

System Programming Project 3

담당 교수 : 김영재

이름 : 김리나

학번 : 20221531

1. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술.
- (주식 서버를 만드는 전체적인 개요에 대해서 작성하면 됨.)

본 프로젝트의 개발 목표는 여러 클라이언트의 동시 접속 요청을 처리할 수 있는 Concurrent Stock Server를 구현하는 것이다. 이를 통해 동시성 개념을 실제 네트워크 서버 개발에 적용해보고, 이벤트 기반과 스레드 기반 두 가지 접근 방식의 차이를 비교하며 성능을 분석한다.

2. 개발 범위 및 내용

A. 개발 범위

- 아래 항목을 구현했을 때의 결과를 간략히 서술

1. Task 1: Event-driven Approach

select() 함수를 이용하여 I/O Multiplexing 기반의 동시성 주식 서버를 구현하였다.

2. Task 2: Thread-based Approach

Pthread 라이브러리를 사용하여 Master Thread가 클라이언트의 연결을 수락하고, Worker Thread Pool이 클라이언트 요청을 병렬적으로 처리하도록 설계하였다. 또한 스레드 간의 동기화를 위해 세마포어를 사용하였으며, 노드 단위의 mutex로 fine-grained locking을 구현하였다.

3. Task 3: Performance Evaluation

주어진 multiclient.c를 통해 클라이언트 수를 변경하며 Event-driven과 Thread-based 방식의 처리 시간을 클라이언트 수와 명령어 종류를 변경하면서 성능을 측정하였다.

B. 개발 내용

- 아래 항목의 내용만 서술
- (기타 내용은 서술하지 않아도 됨. 코드 복사 붙여 넣기 금지)

- **Task1 (Event-driven Approach with select())**

- ✓ Multi-client 요청에 따른 I/O Multiplexing 설명

I/O Multiplexing을 기반으로 하기 때문에 각 클라이언트에 대한 컨텍스트 스위칭 없이 효율적으로 처리할 수 있으며, 이로 인해 서버 자원 사용이 절약된다.

- ✓ epoll과의 차이점 서술

epoll에 비해 select()는 한 번에 감지할 수 있는 파일 디스크립터의 수가 제한되며, 매 호출마다 전체 파일 디스크립터를 순회해야 하기 때문에 성능 면에서는 불리할 수 있다.

- **Task2 (Thread-based Approach with pthread)**

- ✓ Master Thread의 Connection 관리

Master Thread는 클라이언트의 연결 요청을 수락한 후, 해당 소켓 디스크립터를 공유 버퍼에 삽입한다.

- ✓ Worker Thread Pool 관리하는 부분에 대해 서술

여러 개의 Worker Thread는 해당 버퍼에서 디스크립터를 꺼내 클라이언트 요청을 병렬적으로 처리한다.

- **Task3 (Performance Evaluation)**

- ✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

성능 평가 지표는 클라이언트 수 증가에 따른 처리 시간의 변화이다.

Gettimeofday() 함수를 사용하여 Event-driven과 Thread-based 방식의 총 처리 시간을 측정하였고, 클라이언트 수를 1~50 범위 내에서 순차적으로 증가시키며 측정하였다.

- ✓ Configuration 변화에 따른 예상 결과 서술

select 기반 Event driven Approach가 Thread based Approach보다 오버헤드가 낮아서 전체적인 성능이 더 좋을 것 같다. Thread based Approach는 처음엔 동시처리율이 낮았다가 클라이언트 개수가 일정 개수가 넘어가면 높아지

다가 일정하게 유지할 것 같다.

C. 개발 방법

- B.의 개발 내용을 구현하기 위해 어느 소스코드에 어떤 요소를 추가 또는 수정할 것인지 설명. (함수, 구조체 등의 구현이나 수정을 서술)

Event-driven Approach의 경우,

Pool 구조체를 정의하여 클라이언트들의 파일 디스크립터, fd_set, rio_t 구조 등을 통합적으로 관리하도록 하였다.

select()를 통해 다수의 클라이언트 요청을 감지하고, 준비된 요청이 있는 경우 해당 요청을 읽어 처리하도록 구현하였다.

clientfd[] 배열을 사용하여 각 클라이언트의 연결 상태를 관리하고, 처리 완료 후에는 해당 소켓을 닫고 배열에서 제거하였다.

stock.txt 파일을 읽어 트리 구조로 메모리에 저장한 후, 각 요청마다 해당 주식 정보를 읽거나 수정하도록 구현함. 주식 출력의 경우 주식 id 순으로 정렬이 안 되어도 원래 저장된 순서대로 출력될 수 있게 따로 배열을 만들어 관리하였다.

서버 종료 시, 업데이트된 주식 재고 상황을 stock.txt에 저장하도록 했다.

Thread-based Approach의 경우,

sbuf_t라는 공유 버퍼 구조체를 정의하여, Master Thread가 클라이언트의 소켓 디스크립터를 이 버퍼에 저장하고, Worker Thread들이 이를 가져가서 처리하도록 구현하였다.

Master Thread는 Accept()를 통해 새로운 연결을 수락하고, 이를 sbuf_insert() 함수를 통해 버퍼에 삽입하였다.

Worker Thread는 sbuf_remove()를 통해 소켓 디스크립터를 가져온 후, 클라이언트 요청을 처리하였다.

각 주식 노드는 item 구조체로 구현되며, sem_t mutex와 readcnt를 사용하여 Reader-Writer 문제 해결을 위한 동기화를 구현하였다.

서버 종료 시, 업데이트된 주식 재고 상황을 stock.txt에 저장하도록 했다.

3. 구현 결과

- 2번의 구현 결과를 간략하게 작성
- 미처 구현하지 못한 부분에 대해선 디자인에 대한 내용도 추가

Event-driven Approach의 경우,

select() 함수와 fd_set을 활용하여, 단일 프로세스 내에서 다수의 클라이언트 요청을 감지하고 처리할 수 있도록 구현하였다.

서버는 클라이언트로부터의 연결 요청과 각 명령어(show, buy, sell, exit)를 비동기적으로 감지하여 순차적으로 처리하였다.

각 클라이언트는 순서대로 처리가 되지만, 전체 서버는 여러 클라이언트 요청을 동시에 감지하여 빠르게 응답할 수 있었다.

Thread-based Approach의 경우,

Master Thread는 클라이언트의 연결 요청을 받아 연결 소켓을 공유 버퍼(sbuf_t)에 저장하며, Worker Thread는 이를 꺼내 클라이언트 요청을 처리한다.

Worker Thread를 미리 생성하여 thread pool 방식으로 동작하도록 하였다.

각 주식 노드에는 mutex와 readcnt를 기반으로 한 fine-grained locking 기법을 적용하여, 동시 다발적인 show/buy/sell 요청에서도 데이터 무결성을 보장하였다.

show 명령은 reader-writer 문제의 readers-preference 방식으로 처리되며, buy 및 sell은 단독 접근을 통해 안전하게 처리된다.

4. 성능 평가 결과 (Task 3)

- 강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)

Event driven과 Thread based 모두 거래하는 stock의 개수는 10개, 클라이언트 당 요청하는 명령 개수 10개로 두고 명령의 종류는 sell, buy, show 세 가지가 있다. Thread based의 Thread 개수는 10개로 하였다.

1부터 50까지 순차적으로 멀티클라이언트 개수를 늘려가면서 소요 시간을 측정했다.

클라이언트 작업이 끝나면 csv 파일에 (멀티클라이언트 수, 소요 시간, 클라이언트 당 요청개수)가 찍히도록 하였다.

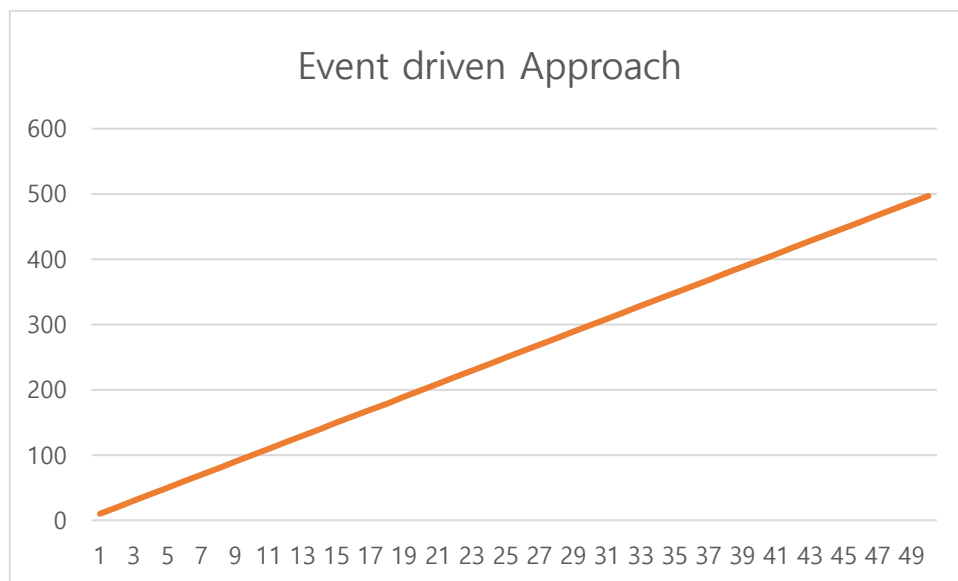
아래에 나오는 그래프들의 가로축과 세로축은 동일하게 가로축은 멀티클라이언트의 개수, 세로축은 동시처리율을 의미한다.

동시처리율은 $(10^8 * \text{multiclient의 개수}) / \text{측정한 시간}$ 으로 구하였다.

성능 측정은 요청 명령어가 랜덤할 때, 특정 명령어만 요청하고 처리할 때(buy), 특정 명령어 두 개를 요청하고 처리할 때(buy/sell), 멀티클라이언트 개수가 백개가 넘어갈 때(10~200) 총 세 경우로 나누어서 진행했다.

```
cse20221531@cspro:~/proj3_task1$ cat output.csv
1,10008681,10
2,10010161,10
3,10012734,10
4,10012328,10
5,10013670,10
6,10014486,10
7,10015574,10
8,10015910,10
9,10016886,10
10,10017176,10
11,10019589,10
12,10020355,10
13,10020535,10
14,10021445,10
15,10023370,10
16,10024964,10
17,10025817,10
18,10081531,10
19,10028449,10
20,10024363,10
21,10029757,10
22,10032120,10
23,10031813,10
24,10033525,10
25,10033753,10
26,10036711,10
27,10035184,10
28,10039415,10
29,10037438,10
30,10036543,10
31,10039205,10
32,10040608,10
33,10042646,10
34,10043407,10
35,10044856,10
36,10043437,10
37,10046989,10
38,10045067,10
39,10048039,10
40,10048740,10
41,10048322,10
42,10048932,10
43,10050227,10
44,10056761,10
45,10052951,10
46,10055463,10
47,10053470,10
48,10054159,10
49,10060395,10
50,10060614,10
```

<Task1 명령어 랜덤 결과 값>



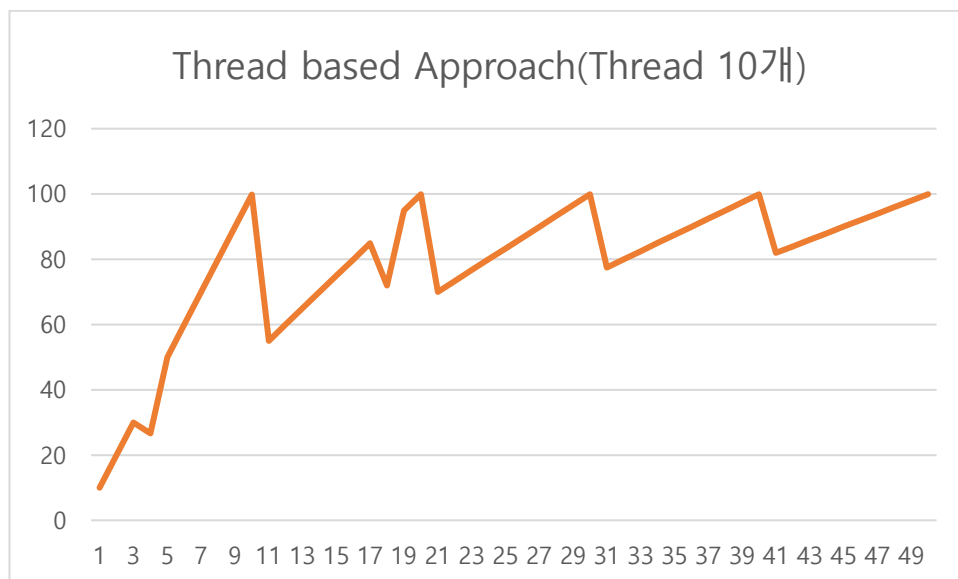
<Event-based Approach(명령어 랜덤)>

```

cse20221531@cspro9:~/proj3_task2$ cat output.csv
1,10009013,10
2,10008704,10
3,10010504,10
4,15016546,10
5,10012456,10
6,10013248,10
7,10017566,10
8,10015250,10
9,10014805,10
10,10016792,10
11,20013544,10
12,20013369,10
13,20014755,10
14,20015340,10
15,20016598,10
16,20018174,10
17,20019053,10
18,25022739,10
19,20019861,10
20,20023121,10
21,30018260,10
22,30018992,10
23,30019804,10
24,30022197,10
25,30021520,10
26,30022959,10
27,30024886,10
28,30023704,10
29,30026893,10
30,30027212,10
31,40022972,10
32,40024073,10
33,40024308,10
34,40025452,10
35,40026656,10
36,40030110,10
37,40028454,10
38,40031054,10
39,40031594,10
40,40031523,10
41,50028082,10
42,50027216,10
43,50029036,10
44,50030730,10
45,50028577,10
46,50029515,10
47,50031327,10
48,50035270,10
49,50035945,10
50,50036714,10

```

<Thread based Approach(Thread 10개) 결과 값>



<Thread based Approach(명령어 랜덤, thread 개수 10개)>

먼저 Task1, 2를 명령이 랜덤하게 요청될 때의 환경에서 처리 소요 시간을 측정하고 성능을 계산하였다.

Task1 : Event-driven의 경우는 예상보다 Thread based 경우보다 훨씬 동시처리율이 높게 나왔고 클라이언트 개수가 증가할수록 선형적으로 증가하였다.

Task2 : Thread based는 동시처리율이 초반엔 낮았다가 멀티클라이언트 수가 증가하면서 동시처리율이 급격히 증가했다가 일정하게 유지되고 있다.

Thread based가 멀티클라이언트 수가 증가하면서 동시처리율이 증가하기는 하지만, Event-driven이 예상보다 훨씬 처리율이 높게 나오는 것에 대해 왜 그런지 생각해 보았다.

이유 1: select()는 context switching이 거의 없음

select()는 하나의 단일 프로세스/스레드가 여러 클라이언트 소켓을 동기적으로 모니터링한다. 따라서 클라이언트 수가 늘어나도 커널 입장에선 하나의 프로세스만 스케줄링해주면 된다.

반면, Thread based은 클라이언트 수만큼의 thread를 만들어야 하므로 context switching이 많아져 오버헤드가 발생한다.

이유 2: 작업량이 작고 요청 간 충돌이 없음

multiclient.c는 대부분 show, buy, sell 요청을 단순 반복한다. 각각의 요청은 매우 간단하고 클라이언트 당 요청하는 명령의 개수도 10개로 적기 때문에

Event-driven 서버가 전체 요청을 순차 처리해도 병목 없이 빠르게 처리한 것 같다.

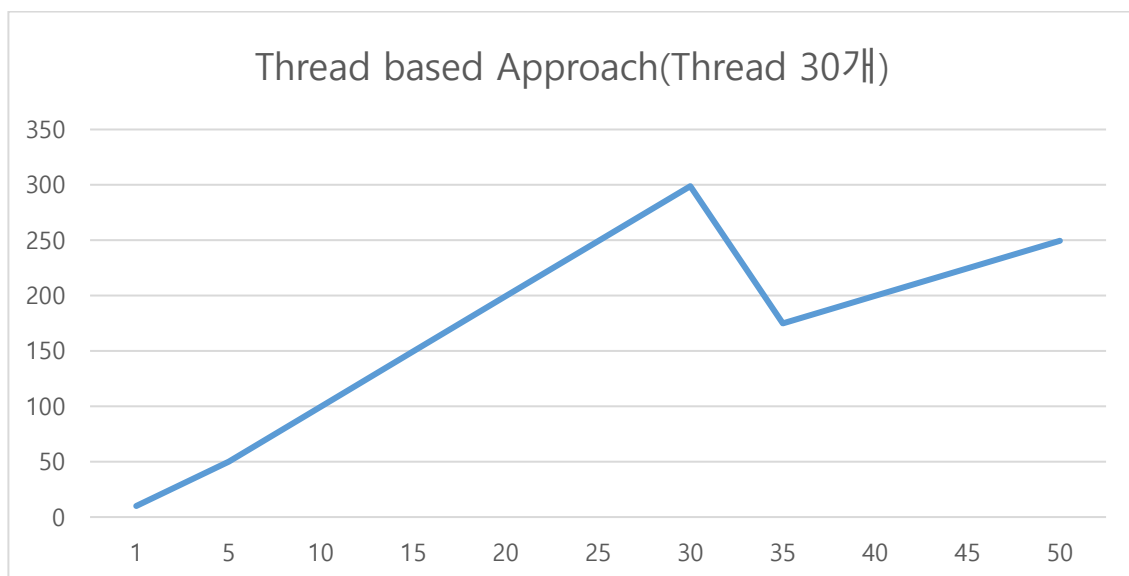
이유 3 : Thread based의 thread의 개수가 10개로 너무 적음

동시처리를 담당하는 thread의 개수를 10개로 두고 측정하였는데 이 수가 너무 적었던 것 같다.

이유 3을 고려하여 thread의 개수를 30개로 다시 define한 뒤 task2를 재측정하였다.

```
cse20221531@csp9:~/proj3_task2$ cat output_thread30.csv
1,10008077,10
5,10011679,10
10,10015611,10
15,10022846,10
20,10026673,10
25,10033282,10
30,10040693,10
35,20017382,10
40,20019871,10
45,20024247,10
50,20033491,10
```

<Thread based Approach(Thread 30개) 결괏값>

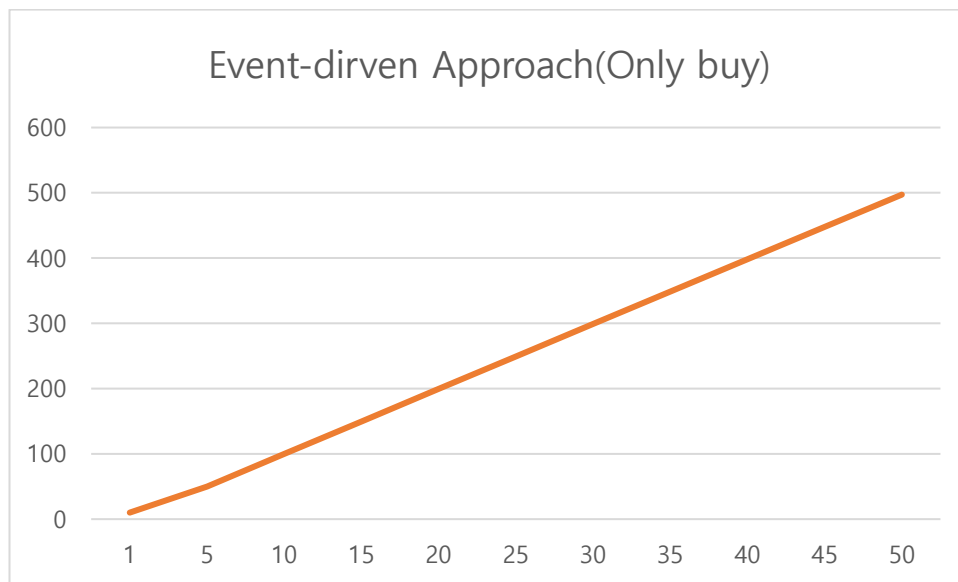


<Thread based Approach(명령어 랜덤, Thread 개수 30개)>

멀티클라이언트 수가 증가할수록 이전에 측정한 thread 10개였을 경우보다 동시처리율이 약 2~3배 증가한 것을 확인할 수 있다.

```
cse20221531@cspro:~/proj3_task1$ cat output_buy.csv
1,10008620,10
5,10012790,10
10,10017081,10
15,10021904,10
20,10027360,10
25,10036023,10
30,10037904,10
35,10044996,10
40,10049422,10
45,10054351,10
50,10056347,10
```

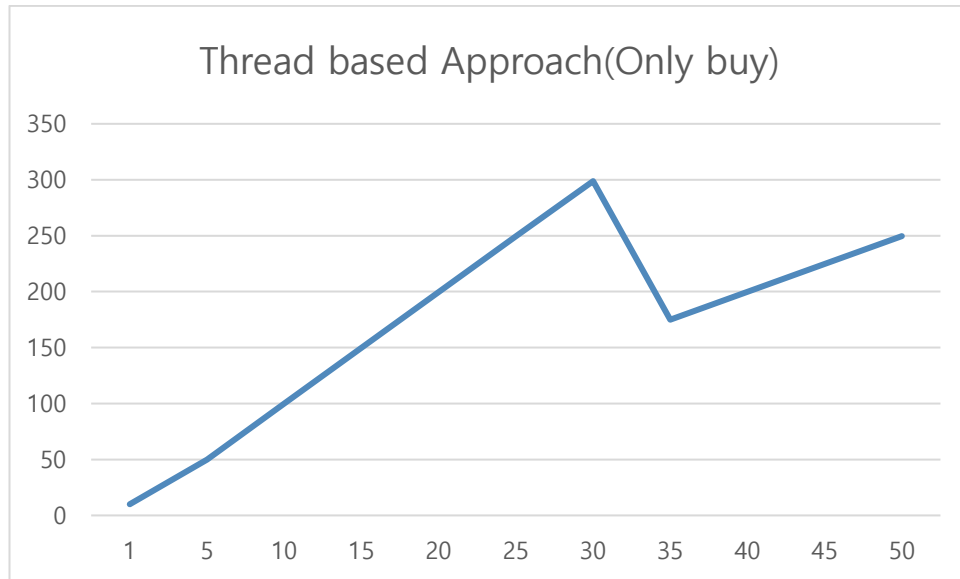
<Event-based Approach(buy 요청만 결괏값)>



<Event-based Approach(buy 요청만)>

```
cse20221531@cspro:~/proj3_task2$ cat task2_buy.csv
1,10008447,10
5,10010975,10
10,10015757,10
15,10022483,10
20,10027387,10
25,10030636,10
30,10040486,10
35,20016866,10
40,20022878,10
45,20025264,10
50,20033740,10
```

<Thread based Approach(buy 요청만 결괏값)>

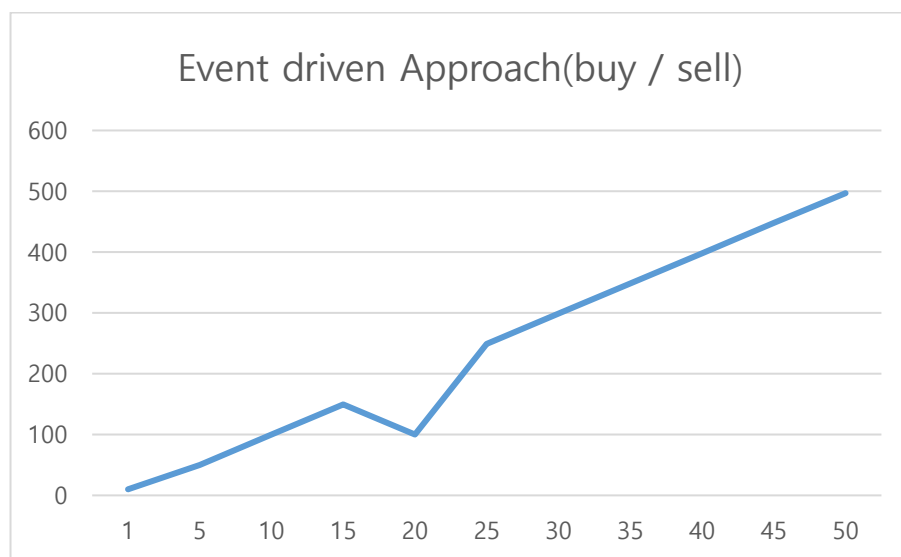


<Thread based Approach(buy 요청만)>

이어서 Task 1,2 모두 buy만 요청하게 하는 환경에서 측정한 결과는 앞선 결과와 거의 동일하게 나와 유의미한 차이를 찾을 수 없었다.

```
cse20221531@csp:~/proj3_task1$ cat task1_buysell.csv
1,10008567,10
5,10012484,10
10,10016571,10
15,10023618,10
20,20039281,10
25,10032483,10
30,10039170,10
35,10044841,10
40,10046500,10
45,10046492,10
50,10063357,10
```

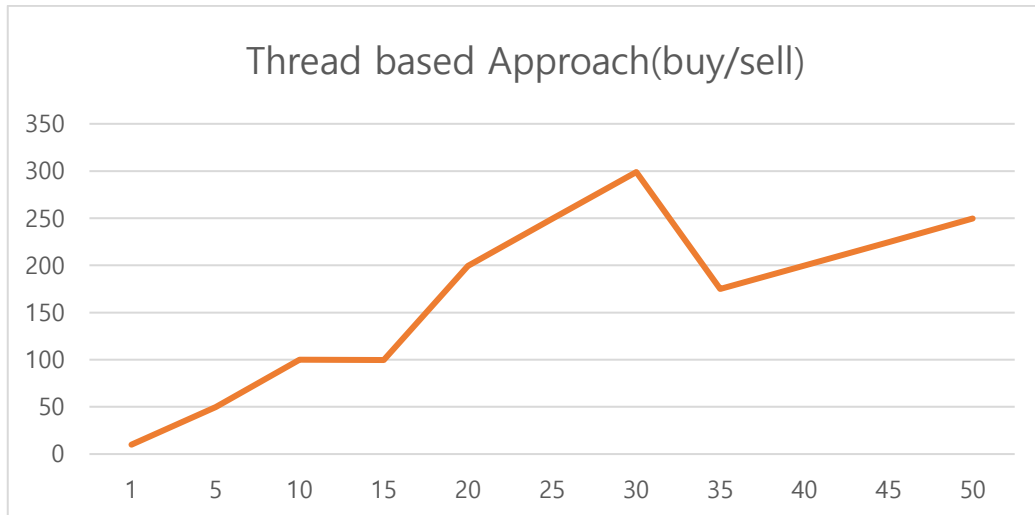
<Event driven Approach(buy와 sell 명령어 섞어서) 결괏값>



<Event driven Approach(buy와 sell 명령어 섞어서)>

```
cse20221531@csp:~/proj3_task2$ cat task2_buysell.csv
1,10008935,10
5,10011659,10
10,10015926,10
15,15027991,10
20,10027568,10
25,10031840,10
30,10037766,10
35,20014674,10
40,20022604,10
45,20028260,10
50,20033360,10
```

Tread based Approach(buy와 sell 명령어 섞어서) 결괏값>

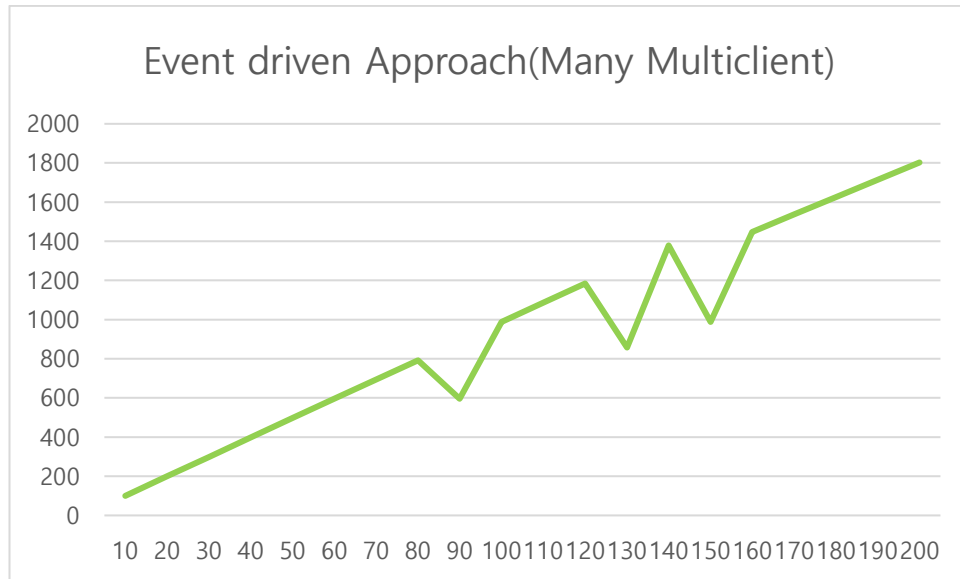


<Tread based Approach(buy와 sell 명령어 섞어서)>

buy와 sell을 섞어서 요청했을 때도 전체적인 추세를 보면 앞선 결과들과 비슷하게 나왔다.

```
cse20221531@cspro:~/proj3_task1$ cat task1_to200.csv
10,10018941,10
20,10029152,10
30,10037392,10
40,10044406,10
50,10059825,10
60,10066276,10
70,10083876,10
80,10094669,10
90,15109415,10
100,10111786,10
110,10125751,10
120,10126719,10
130,15156109,10
140,10158229,10
150,15176628,10
160,11047614,10
170,11057300,10
180,11072406,10
190,11079304,10
200,11094847,10
```

<Event driven Approach(Multiclient 100개 이상)>



<Event driven Approach(Multiclient 100개 이상)>

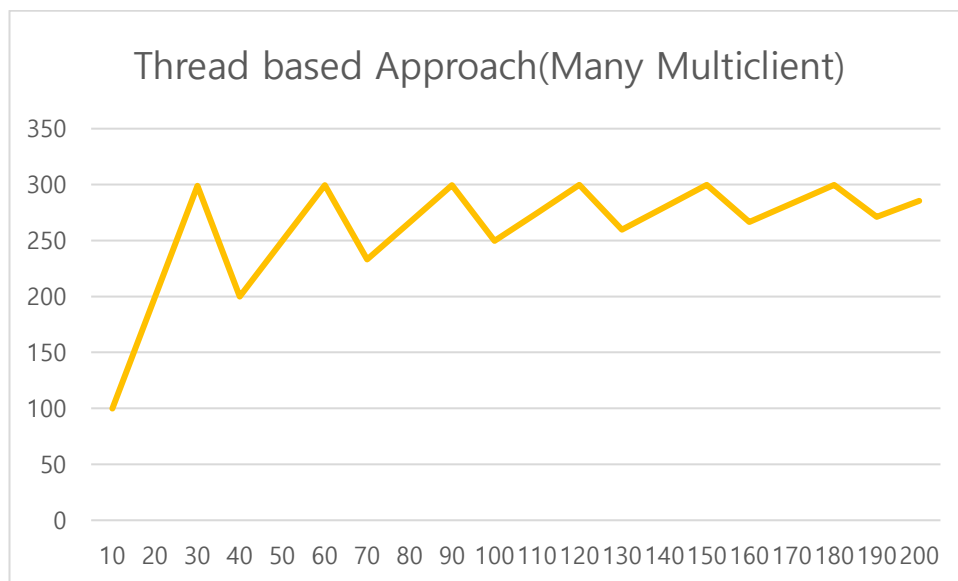
멀티클라이언트가 100개가 넘어가는 경우는 멀티클라이언트의 개수를 10부터 시작해 10 단위로 200개까지 늘려가며 시간을 측정하였다.

Task1은 50개 이하였을 때와 마찬가지로 동시처리율이 선형적으로 증가하는 건 같았지만

90개가 넘어가는 이후부터는 처리가 일시적으로 지연되면서 시간이 증가하는 부분을 관측할 수 있었다.

```
cse20221531@cspro:~/proj3_task2$ cat task2_to200.csv
10,10016291,10
20,10026978,10
30,10037063,10
40,20019824,10
50,20031079,10
60,20041140,10
70,30027117,10
80,30034549,10
90,30047848,10
100,40031361,10
110,40041346,10
120,40047901,10
130,50036431,10
140,50046637,10
150,50056150,10
160,60039385,10
170,60052533,10
180,60060728,10
190,70047697,10
200,70056743,10
```

<Thread based Approach(Multiclient 100개 이상)>



<Thread based Approach(Multiclient 100개 이상)>

Task2로 멀티클라이언트가 100개가 넘어가는 경우를 측정한 결과값을 보았을 때 확인할 수 있는 부분은 30개 단위로 소요 시간이 거의 비슷하다는 것이었다. 그 이유는 Thread의 개수가 30개여서 이러한 결과가 나오는 것 같다.

동시처리율을 나타낸 그래프를 보면 멀티클라이언트의 개수가 증가함에 따라 동시처

리울이 진동하면서 수렴하는 걸 확인할 수 있다.